Chapter 15 Review Sheet

15.1 Processors and parallel architectures

Feature asked in the tick table	RISC	CISC
Instruction length	Fixed	Variable
Time for one instruction	One clock cycle	Several clock cycles
General-purpose registers	Many	Fewer
How an instruction uses memory	Load and store instructions only	Many addressing modes; an instruction can access memory itself
Control unit	Hardwired	Microprogrammed
Pipelining	Used. Fixed length makes the stages line up	Harder, because instructions differ in length and in time

Many registers: the operands come from registers, so the instruction does not have to wait for memory and can finish in one cycle.

Pipelining

- The instruction is split into stages. Name two of: fetch, decode, execute, write-back.
- Several instructions occupy different stages at the same time.
- Each stage takes one clock cycle.
- Once every stage is busy, one instruction finishes on each clock cycle. Throughput rises. A single instruction does not get shorter.

When an interrupt arrives

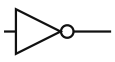
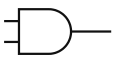
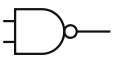
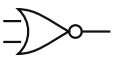
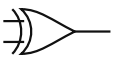
- It is checked at the end of the current instruction. On RISC that is one cycle, so it is taken quickly. On CISC the instruction may still have many cycles left, so the interrupt waits.
- Later instructions are already in the pipeline. Flush those stages, or save the contents of each stage. The instruction in write-back can finish.
- Save the program counter and the registers, run the interrupt service routine, restore them, then fill the pipeline again.
- Frequent interrupts keep emptying the pipeline, so the throughput gain is lost.

	Instruction streams	Data
SISD	One processor runs one stream	One data stream
SIMD	Many processors run the same instruction	Each has its own data stream
MISD	Many processors run different instructions	All use the same data stream
MIMD	Many processors run different instructions	Each has its own data stream

The mark is for what the processors do. In a comparison, name both architectures: SIMD and MISD are both parallel and both use more than one processor; SIMD applies one instruction to many data items, while MISD applies different instructions to one data item. Expanding the initials on their own scores nothing.

Massively parallel. A very large number of processors are linked and act as one machine (“many” or “thousands”; “several” does not score). They work at the same time on one problem, each on its own part. They exchange data through an interconnection network, or through shared memory.

15.2 Gates used in every circuit question

NOT	AND	OR	NAND	NOR	XOR
					
$\overline{A}$ 0 gives 1, 1 gives 0	A.B	A + B	$\overline{A.B}$	$\overline{A + B}$	A ⊕ B

A	B	AND	OR	NAND	NOR	XOR
0	0	0	0	1	1	0
0	1	0	1	1	0	1
1	0	0	1	1	0	1
1	1	1	1	0	0	0

On a circuit question, add a working column for each gate before the output column. Those columns carry marks of their own.

Chapter 15 Review Sheet

Adders

Half adder — two bits, no carry in

Sum = A ⊕ B Carry = A.B

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

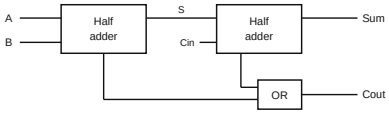
One XOR gate makes the sum. One AND gate makes the carry. It can start a binary addition. It cannot take a carry from the previous column.

Full adder — A, B and a carry in

Sum = A ⊕ B ⊕ Cin

Cout = A.B + Cin.(A ⊕ B)

The same Cout is A.B + A.Cin + B.Cin. It is 1 when at least two of the three inputs are 1. Sum is 1 when an odd number of inputs are 1.



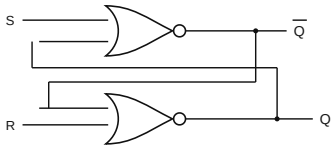
Two half adders, then OR the two carry outputs. Cout of one full adder is Cin of the next bit.

Flip-flops — each one stores one bit

SR, built from cross-coupled NOR gates

S enters the gate whose output is $\overline{Q}$. R enters the gate whose output is Q. Each output is fed back into the other gate.

S	R	Q next	What to write
0	0	unchanged	Holds the stored bit
0	1	0	Reset. $\overline{Q} = 1$
1	0	1	Set. $\overline{Q} = 0$
1	1	invalid	Both outputs are forced to 0, so $\overline{Q}$ is not the opposite of Q. When S and R return to 0, the bit is unpredictable.



Top NOR output is $\overline{Q}$. Bottom NOR output is Q. The wire from each output is an input of the other gate.

JK — clocked, and no invalid input

J	K	Q after the clock
0	0	Q, no change
0	1	0, reset
1	0	1, set
1	1	$\overline{Q}$, toggle

Outputs change on the clock pulse. J = K = 1 flips the stored bit. That is the input an SR flip-flop cannot accept.

Boolean laws — name the law in the working

The final line on its own does not get the method marks. Each different law used earns a mark.

Law to name	The step that scores
De Morgan	$\overline{A.B} = \overline{A} + \overline{B}$ and $\overline{A + B} = \overline{A} \,.\, \overline{B}$. Break the bar and change the operator.
Double complement	NOT(NOT A) = A. A second bar cancels the first.
Distributive	A + B.C = (A + B)(A + C) and A.(B + C) = A.B + A.C
Absorption	A + A.B = A and A.(A + B) = A
Complement	$A + \overline{A} = 1$ and $A.\overline{A} = 0$
Identity	A + 0 = A and A.1 = A
Annulment	A + 1 = 1 and A.0 = 0
Idempotent	A + A = A and A.A = A

Karnaugh maps and a sum of products

Truth table to sum of products

- One product for every row in which the output is 1.
- Every input appears in every product. Complement an input that is 0 in that row.
- Join the products with +. Leave it unsimplified until a later part asks.
- The mark is usually one for every two correct products.

Reading the loops

- Headings run 00, 01, 11, 10 on both edges. One bit changes between neighbours, and the map wraps: the first cell is next to the last.
- For three variables the single variable labels one side and the pair labels the other, still in that order. Use the labels printed on the paper.
- Fill every cell. On a two-mark map, one wrong cell loses one mark and two wrong cells lose both.
- Stop if the question says not to simplify further. The next part is the factorised form, for example A.C + B.C = C.(A + B), and that step is marked on its own.

	CD			
AB	00	01	11	10
00				
01				
11				
10				

11 sits between 01 and 10. The row 00 is adjacent to the row 10, and the column 00 is adjacent to the column 10.

- Loop the 1s in a rectangle of 1, 2, 4, 8 or 16 cells. Make the loops as large as possible and use as few as possible. Loops may overlap and may leave one edge and continue on the opposite edge.
- A group of 3 or 6, or a diagonal, does not score. An incorrect loop can cap the marks for that part. One correct loop earns one mark.
- From a loop, keep a variable that stays the same: write it when it is 1, complement it when it is 0, and drop it when it changes. Join the products with +.